

Overview of Text summarization

Subject: Text summarization

1.

Recent approaches Recently the rise of transformer models replacing more traditional RNN (LSTM) have provided a flexibility in the mapping of text sequences to text sequences of a different type, which is well suited to automatic summarization. This includes models such as T5 and Pegasus.

2.

Intrinsic versus extrinsic Intrinsic evaluation assesses the summaries directly, while extrinsic evaluation evaluates how the summarization system affects the completion of some other task. Intrinsic evaluations have assessed mainly the coherence and informativeness of summaries. Extrinsic evaluations, on the other hand, have tested the impact of summarization on tasks like relevance assessment, reading comprehension, etc.

3.

Submodular functions as generic tools for summarization The idea of a submodular set function has recently emerged as a powerful modeling tool for various summarization problems. Submodular functions naturally model notions of coverage, information, representation and diversity. Moreover, several important combinatorial optimization problems occur as special instances of submodular optimization. For example, the set cover problem is a special case of submodular optimization, since the set cover function is submodular. The set cover function attempts to find a subset of objects which cover a given set of concepts. For example, in document summarization, one would like the summary to cover all important and relevant concepts in the document. This is an instance of set cover. Similarly, the facility location problem is a special case of submodular functions. The Facility Location function also naturally models coverage and diversity. Another example of a submodular optimization problem is using a determinantal point process to model diversity. Similarly, the Maximum-Marginal-Relevance procedure can also be seen as an instance of submodular optimization. All these important models encouraging coverage, diversity and information are all submodular. Moreover, submodular functions can be efficiently combined, and the resulting function is still submodular. Hence, one could combine one submodular function which models diversity, another one which models coverage and use human supervision to learn a right model of a submodular function for the problem. While submodular functions are fitting problems for summarization, they also admit very efficient algorithms for optimization. For example, a simple greedy algorithm admits a constant factor guarantee. Moreover, the greedy algorithm is extremely simple to implement and can scale to large datasets, which is very important for summarization problems. Submodular functions have achieved state-of-the-art for almost all summarization problems. For example, work by Lin and Bilmes, 2012 shows that submodular functions achieve the best results to date on DUC-04, DUC-05, DUC-06 and DUC-07 systems for document summarization. Similarly, work by Lin and Bilmes, 2011, shows that many existing systems for automatic summarization are instances of submodular functions. This was a breakthrough result establishing submodular functions as the right models for summarization problems. Submodular Functions have also been used for other summarization tasks. Tschitschek et al., 2014 show that mixtures of submodular functions achieve state-of-the-art results for image collection

summarization. Similarly, Bairi et al., 2015 show the utility of submodular functions for summarizing multi-document topic hierarchies. Submodular Functions have also successfully been used for summarizing machine learning datasets.

4.

Supervised learning approaches Beginning with the work of Turney, many researchers have approached keyphrase extraction as a supervised machine learning problem. Given a document, we construct an example for each unigram, bigram, and trigram found in the text (though other text units are also possible, as discussed below). We then compute various features describing each example (e.g., does the phrase begin with an upper-case letter?). We assume there are known keyphrases available for a set of training documents. Using the known keyphrases, we can assign positive or negative labels to the examples. Then we learn a classifier that can discriminate between positive and negative examples as a function of the features. Some classifiers make a binary classification for a test example, while others assign a probability of being a keyphrase. For instance, in the above text, we might learn a rule that says phrases with initial capital letters are likely to be keyphrases. After training a learner, we can select keyphrases for test documents in the following manner. We apply the same example-generation strategy to the test documents, then run each example through the learner. We can determine the keyphrases by looking at binary classification decisions or probabilities returned from our learned model. If probabilities are given, a threshold is used to select the keyphrases. Keyphrase extractors are generally evaluated using precision and recall. Precision measures how many of the proposed keyphrases are actually correct. Recall measures how many of the true keyphrases your system proposed. The two measures can be combined in an F-score, which is the harmonic mean of the two ($F = 2PR/(P + R)$). Matches between the proposed keyphrases and the known keyphrases can be checked after stemming or applying some other text normalization. Designing a supervised keyphrase extraction system involves deciding on several choices (some of these apply to unsupervised, too). The first choice is exactly how to generate examples. Turney and others have used all possible unigrams, bigrams, and trigrams without intervening punctuation and after removing stopwords. Hulth showed that you can get some improvement by selecting examples to be sequences of tokens that match certain patterns of part-of-speech tags. Ideally, the mechanism for generating examples produces all the known labeled keyphrases as candidates, though this is often not the case. For example, if we use only unigrams, bigrams, and trigrams, then we will never be able to extract a known keyphrase containing four words. Thus, recall may suffer. However, generating too many examples can also lead to low precision. We also need to create features that describe the examples and are informative enough to allow a learning algorithm to discriminate keyphrases from non-keyphrases. Typically features involve various term frequencies (how many times a phrase appears in the current text or in a larger corpus), the length of the example, relative position of the first occurrence, various Boolean syntactic features (e.g., contains all caps), etc. The Turney paper used about 12 such features. Hulth uses a reduced set of features, which were found most successful in the KEA (Keyphrase Extraction Algorithm) work derived from Turney's seminal paper. In the end, the system will need to return a list of keyphrases for a test document, so we need to have a way to limit the number. Ensemble methods (i.e., using votes from several classifiers) have been used to produce numeric scores that can be thresholded to provide a user-provided number of keyphrases. This is the technique used by Turney with C4.5 decision trees. Hulth used a single binary classifier so the learning algorithm implicitly determines the appropriate number. Once examples and features are created, we need a way to learn to predict keyphrases. Virtually any supervised learning algorithm could be used, such as decision trees, Naive Bayes, and

rule induction. In the case of Turney's GenEx algorithm, a genetic algorithm is used to learn parameters for a domain-specific keyphrase extraction algorithm. The extractor follows a series of heuristics to identify keyphrases. The genetic algorithm optimizes parameters for these heuristics with respect to performance on training documents with known key phrases.